

LAPORAN PRAKTIKUM PEKAN 8

STRUKTUR DATA



Disusun oleh:

FAHRI SYAH PUTRA RAMADHAN

2511531015

Dosen pengampu:

Dr. Wahyudi. MT

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG, 2026

PENJELASAN KODE PROGRAM

QUICKSORT_2511531015

Program ini merupakan implementasi algoritma **Quick Sort** menggunakan bahasa pemrograman Java. Quick Sort adalah salah satu algoritma pengurutan data yang menggunakan konsep **divide and conquer**, yaitu membagi data menjadi beberapa bagian kecil, kemudian mengurutkan setiap bagian tersebut secara rekursif.

Pada program ini, data yang akan diurutkan disimpan dalam array bertipe integer. Data awal yang digunakan adalah:

```
int[] arr = {10, 7, 8, 9, 1, 5};
```

Program akan mengurutkan data tersebut dari nilai terkecil ke nilai terbesar menggunakan metode Quick Sort. Selain itu, program ini juga menggunakan teknik **Median of Three** untuk menentukan pivot agar proses pengurutan menjadi lebih baik.

1. Deklarasi Package dan Class

```
package pekan8_2511531015;
```

```
public class quicksort_2511531015 {
```

Bagian ini menunjukkan bahwa program berada di dalam package pekan8_2511531015. Nama class yang digunakan adalah quicksort_2511531015. Penambahan angka 2511531015 menunjukkan identitas atau NIM yang digunakan dalam penamaan class sesuai kebutuhan praktikum.

2. Method swap_1015

```
static void swap_1015(int[] arr, int i, int j) {  
    int temp = arr[i];  
    arr[i] = arr[j];  
    arr[j] = temp;  
}
```

Method swap_1015 digunakan untuk menukar posisi dua elemen dalam array. Method ini menerima tiga parameter, yaitu array arr, indeks pertama i, dan indeks kedua j.

Proses pertukaran dilakukan dengan bantuan variabel sementara temp. Nilai pada indeks i disimpan terlebih dahulu ke dalam temp, kemudian nilai pada indeks j dipindahkan ke indeks i, dan terakhir nilai dari temp dimasukkan ke indeks j. Contohnya, jika terdapat array:

10 7 8

lalu elemen indeks 0 dan indeks 1 ditukar, maka hasilnya menjadi:

3. Method MedianOfThree_1015

```
static void MedianOfThree_1015(int[] arr, int low, int high) {
    int mid = low + (high - low) / 2;
```

Method ini digunakan untuk menentukan pivot dengan teknik **Median of Three**. Teknik ini mengambil tiga elemen sebagai pembanding, yaitu elemen pertama, elemen tengah, dan elemen terakhir dari bagian array yang sedang diproses.

```
if (arr[low] > arr[mid]) {
    swap_1015(arr, low, mid);
}
```

```
if (arr[low] > arr[high]) {
    swap_1015(arr, low, high);
}
```

```
if (arr[mid] > arr[high]) {
    swap_1015(arr, mid, high);
}
```

Bagian ini bertujuan untuk mengurutkan tiga elemen tersebut, yaitu arr[low], arr[mid], dan arr[high], sehingga nilai tengahnya dapat dijadikan pivot.

```
swap_1015(arr, mid, high);
```

Setelah diperoleh nilai median, nilai tersebut dipindahkan ke posisi high. Dengan begitu, elemen pada indeks high akan digunakan sebagai pivot pada proses partisi.

Penggunaan Median of Three bertujuan untuk mengurangi kemungkinan pemilihan pivot yang buruk, terutama jika data sudah hampir terurut atau memiliki pola tertentu.

4. Method partition_1015

```
static int partition_1015(int[] arr, int low, int high) {
    MedianOfThree_1015(arr, low, high);
```

Method partition_1015 digunakan untuk membagi array menjadi dua bagian berdasarkan nilai pivot. Sebelum proses partisi dilakukan, method MedianOfThree_1015 dipanggil terlebih dahulu untuk menentukan pivot.

```
int pivot = arr[high];
int i = low - 1;
```

Variabel pivot berisi nilai pivot yang berada pada posisi high. Variabel i digunakan sebagai penanda posisi terakhir dari elemen yang lebih kecil dari pivot.

```
for (int j = low; j <= high - 1; j++) {  
    if (arr[j] < pivot) {  
        i++;  
        swap_1015(arr, i, j);  
    }  
}
```

Perulangan ini digunakan untuk memeriksa setiap elemen dari indeks low sampai high - 1. Jika nilai elemen lebih kecil dari pivot, maka elemen tersebut akan dipindahkan ke bagian kiri array dengan cara ditukar menggunakan method swap_1015.

```
swap_1015(arr, i + 1, high);  
return i + 1;
```

Setelah semua elemen dibandingkan, pivot diletakkan pada posisi yang benar, yaitu di antara elemen-elemen yang lebih kecil dan elemen-elemen yang lebih besar. Method ini kemudian mengembalikan posisi akhir pivot.

5. Method quickSort_1015

```
static void quickSort_1015(int[] arr, int low, int high) {  
    if (low < high) {  
        int pi = partition_1015(arr, low, high);
```

Method quickSort_1015 adalah method utama dalam proses pengurutan Quick Sort. Method ini bekerja secara rekursif. Jika indeks low masih lebih kecil dari high, maka array masih perlu diurutkan.

```
quickSort_1015(arr, low, pi - 1);  
quickSort_1015(arr, pi + 1, high);
```

Setelah proses partisi dilakukan, array dibagi menjadi dua bagian. Bagian kiri berisi elemen-elemen yang lebih kecil dari pivot, sedangkan bagian kanan berisi elemen-elemen yang lebih besar dari pivot.

Kemudian method quickSort_1015 dipanggil kembali untuk mengurutkan bagian kiri dan bagian kanan. Proses ini terus berulang sampai seluruh bagian array terurut.

6. Method printArr_1015

```
public static void printArr_1015(int[] arr) {  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");
```

```
}  
  
    System.out.println();  
}
```

Method `printArr_1015` digunakan untuk menampilkan isi array ke layar. Method ini menggunakan perulangan `for` untuk mencetak setiap elemen array dari indeks pertama sampai indeks terakhir.

Setelah semua elemen dicetak, program menjalankan `System.out.println()` agar output berikutnya ditampilkan pada baris baru.

7. Method main

```
public static void main(String[] args) {  
    int[] arr = {10, 7, 8, 9, 1, 5};  
    int N = arr.length;
```

Method `main` adalah bagian utama yang pertama kali dijalankan saat program dieksekusi. Pada bagian ini, array berisi data awal dideklarasikan. Variabel `N` digunakan untuk menyimpan panjang array.

```
    System.out.print("Data sebelum diurutkan: ");  
    printArr_1015(arr);
```

Bagian ini menampilkan data sebelum dilakukan proses pengurutan.

```
    quickSort_1015(arr, 0, N - 1);
```

Perintah ini memanggil method `quickSort_1015` untuk mengurutkan array mulai dari indeks 0 sampai indeks terakhir, yaitu `N - 1`.

```
    System.out.print("Data terurut Quick Sort: ");  
    printArr_1015(arr);
```

Setelah proses pengurutan selesai, program menampilkan array yang sudah terurut.

Output Program

Jika program dijalankan, maka output yang dihasilkan adalah:

Data sebelum diurutkan: 10 7 8 9 1 5

Data terurut Quick Sort: 1 5 7 8 9 10

```
Data sebelum diurutkan: 10 7 8 9 1 5  
Data terurut Quick Sort: 1 5 7 8 9 10
```

SHELLSORT_2511531015

Shell Sort adalah pengembangan dari algoritma **Insertion Sort**. Perbedaannya, Shell Sort tidak langsung membandingkan elemen yang bersebelahan, tetapi membandingkan elemen dengan jarak tertentu yang disebut **gap**. Nilai gap akan terus diperkecil sampai akhirnya menjadi 1.

Pada program ini, data yang akan diurutkan disimpan dalam array bertipe integer, yaitu:

```
int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
```

Tujuan dari program ini adalah mengurutkan data tersebut dari nilai terkecil sampai nilai terbesar.

1. Deklarasi Package dan Class

```
package pekan8_2511531015;
```

```
public class shellsort_2511531015 {
```

Bagian ini menunjukkan bahwa program berada di dalam package pekan8_2511531015. Nama class yang digunakan adalah shellsort_2511531015. Penambahan angka 2511531015 pada nama package dan class menunjukkan identitas atau NIM yang digunakan dalam praktikum.

2. Method shellsort_2511531015

```
public static void shellsort_2511531015(int[] A) {
```

Method ini digunakan untuk melakukan proses pengurutan data menggunakan algoritma Shell Sort. Parameter int[] A merupakan array yang akan diurutkan.

```
int n_1015 = A.length;
```

```
int gap_1015 = n_1015/2;
```

Variabel n_1015 digunakan untuk menyimpan panjang array. Sedangkan variabel gap_1015 digunakan untuk menentukan jarak antar elemen yang akan dibandingkan. Nilai awal gap diperoleh dari panjang array dibagi dua.

Pada data yang digunakan, jumlah elemen array adalah 10, sehingga nilai awal gap adalah:

```
gap = 10 / 2 = 5
```

3. Perulangan Selama Gap Lebih Besar dari 0

```
while (gap_1015 > 0) {
```

Perulangan while ini digunakan untuk menjalankan proses pengurutan selama nilai gap_1015 masih lebih besar dari 0. Setelah satu proses selesai, nilai gap akan dibagi dua sampai akhirnya menjadi 0. Contoh perubahan nilai gap pada data berjumlah 10 adalah:

5, 2, 1, 0

Ketika gap bernilai 1, proses pengurutan akan berjalan seperti Insertion Sort biasa.

4. Perulangan untuk Membandingkan Elemen

```
for (int i = gap_1015; i < n_1015; i++) {
```

```
    int temp = A[i];
```

```
    int j = i;
```

Perulangan for digunakan untuk menelusuri elemen array mulai dari indeks sebesar nilai gap. Variabel temp digunakan untuk menyimpan sementara nilai elemen yang sedang diperiksa. Sedangkan variabel j digunakan sebagai indeks untuk proses pergeseran elemen.

Sebagai contoh, jika nilai gap adalah 5, maka elemen pada indeks ke-5 akan dibandingkan dengan elemen pada indeks ke-0. Setelah itu indeks ke-6 dibandingkan dengan indeks ke-1, dan seterusnya.

5. Proses Pergeseran Elemen

```
while (j >= gap_1015 && A[j - gap_1015] > gap_1015) {
```

```
    A[j] = A[j - gap_1015];
```

```
    j = j - gap_1015; }
```

Bagian ini digunakan untuk membandingkan elemen sebelumnya yang berjarak sebesar nilai gap. Jika elemen sebelumnya lebih besar, maka elemen tersebut akan digeser ke posisi kanan.

Namun, pada kode ini terdapat bagian yang kurang tepat, yaitu:

```
A[j - gap_1015] > gap_1015
```

Pada Shell Sort, seharusnya elemen sebelumnya dibandingkan dengan nilai sementara temp, bukan dibandingkan dengan nilai gap_1015. Jadi bentuk yang lebih tepat adalah:

```
A[j - gap_1015] > temp
```

Karena gap_1015 hanya berfungsi sebagai jarak antar elemen, bukan nilai pembanding data.

6. Menempatkan Nilai Sementara ke Posisi yang Tepat

```
A[j] = temp;
```

Setelah proses pergeseran selesai, nilai yang sebelumnya disimpan di variabel temp dimasukkan kembali ke posisi yang sesuai. Dengan demikian, elemen tersebut ditempatkan pada urutan yang benar berdasarkan nilai gap saat itu.

7. Mengurangi Nilai Gap

```
gap_1015 = gap_1015 / 2;
```

Setelah satu tahap pengurutan selesai, nilai gap akan dibagi dua. Proses ini dilakukan berulang sampai gap bernilai 0. Ketika gap sudah 0, proses pengurutan berhenti.

8. Method main

```
public static void main(String[] args) {
```

Method main adalah method utama yang pertama kali dijalankan saat program dieksekusi.

```
int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5,};
```

Bagian ini digunakan untuk mendeklarasikan array yang berisi data awal yang akan diurutkan.

```
System.out.print("Sebelum:");  
printArray(data);
```

Perintah ini digunakan untuk menampilkan data sebelum dilakukan proses pengurutan.

```
shellsort_2511531015(data);
```

Perintah ini digunakan untuk memanggil method shellsort_2511531015, sehingga array data akan diproses menggunakan algoritma Shell Sort.

```
System.out.print("sesudah (shellsort): ");  
printArray(data);
```

Setelah proses pengurutan selesai, data yang sudah diurutkan akan ditampilkan ke layar.

9. Method printArray

```
public static void printArray(int[] arr) {  
    for (int i_1015 : arr) System.out.print(i_1015 + " ");  
    System.out.println(); }  

```

Method printArray digunakan untuk menampilkan isi array. Perulangan for-each digunakan untuk mengambil setiap elemen di dalam array, kemudian mencetaknya satu per satu ke layar.

Variabel i_1015 berfungsi untuk menyimpan nilai elemen array yang sedang dibaca.

```
Sebelum: 3 10 4 6 8 9 7 2 1 5  
sesudah (shellsort): 1 7 3 9 4 8 5 2 10 6
```

MERGESORT_2511531015

Merge Sort adalah algoritma pengurutan data yang menggunakan konsep **divide and conquer**, yaitu membagi data menjadi beberapa bagian kecil, mengurutkan setiap bagian, kemudian menggabungkannya kembali menjadi satu array yang sudah terurut.

Pada program ini, data yang akan diurutkan disimpan dalam array bertipe integer, yaitu:

```
int arr[] = {12, 11, 13, 5, 6, 7};
```

Tujuan dari program ini adalah mengurutkan data tersebut dari nilai terkecil hingga nilai terbesar.

1. Deklarasi Package dan Class

```
package pekan8_2511531015;
```

```
public class mergeSort_2511531015 {
```

Bagian ini menunjukkan bahwa program berada di dalam package `pekan8_2511531015`. Nama class yang digunakan adalah `mergeSort_2511531015`. Penambahan angka 2511531015 menunjukkan identitas atau NIM yang digunakan dalam praktikum.

2. Method merge_1015

```
void merge_1015(int arr[], int l, int m, int r) {
```

Method `merge_1015` berfungsi untuk menggabungkan dua bagian array yang sudah terurut. Parameter `arr[]` adalah array utama, `l` adalah indeks kiri, `m` adalah indeks tengah, dan `r` adalah indeks kanan.

```
int n1 = m - l + 1;
```

```
int n2 = r - m;
```

Variabel `n1` digunakan untuk menentukan ukuran subarray kiri, sedangkan `n2` digunakan untuk menentukan ukuran subarray kanan.

```
int L[] = new int[n1];
```

```
int R[] = new int[n2];
```

Bagian ini membuat dua array sementara, yaitu `L[]` dan `R[]`. Array `L[]` digunakan untuk menyimpan bagian kiri, sedangkan `R[]` digunakan untuk menyimpan bagian kanan.

```
for (int i = 0; i < n1; i++) {
```

```
    L[i] = arr[l + i];}
```

Perulangan ini digunakan untuk menyalin data dari array utama ke array sementara `L[]`.

```
for (int j = 0; j < n2; j++) {
```

```
R[j] = arr[m + 1 + j]; }
```

Perulangan ini digunakan untuk menyalin data dari array utama ke array sementara R[].

Setelah data disalin, proses penggabungan dilakukan dengan membandingkan elemen dari array L[] dan R[].

```
int i = 0;
```

```
int j = 0;
```

```
int k = 1;
```

Variabel i digunakan sebagai indeks untuk array kiri L[], variabel j digunakan sebagai indeks untuk array kanan R[], dan variabel k digunakan sebagai indeks untuk array utama arr[].

```
while (i < n1 && j < n2) {
```

```
    if (L[i] <= R[j]) {
```

```
        arr[k] = L[i];
```

```
        i++;
```

```
    } else {
```

```
        arr[k] = R[j];
```

```
        j++;
```

```
    }
```

```
    k++; }
```

Bagian ini membandingkan elemen pada array L[] dan R[]. Jika elemen pada L[] lebih kecil atau sama dengan elemen pada R[], maka elemen tersebut dimasukkan ke array utama. Jika tidak, elemen dari R[] yang dimasukkan ke array utama.

Dengan proses ini, elemen-elemen akan digabungkan kembali dalam urutan yang benar.

```
while (i < n1) {
```

```
    arr[k] = L[i];
```

```
    i++;
```

```
    k++; }
```

Bagian ini digunakan untuk menyalin sisa elemen dari array L[] jika masih ada elemen yang belum dimasukkan ke array utama.

```
while (j < n2) {
```

```
arr[k] = R[j];  
j++;  
k++; }
```

Bagian ini digunakan untuk menyalin sisa elemen dari array R[] jika masih ada elemen yang belum dimasukkan ke array utama.

3. Method sort_1015

```
void sort_1015(int arr[], int l, int r) {
```

Method sort_1015 merupakan method utama dalam proses Merge Sort. Method ini berfungsi untuk membagi array menjadi bagian-bagian kecil secara rekursif.

```
if (l < r) {
```

Kondisi ini digunakan untuk memastikan bahwa array masih dapat dibagi. Jika indeks kiri l lebih kecil dari indeks kanan r, maka proses pembagian masih dilakukan.

```
int m = (l + r) / 2;
```

Bagian ini digunakan untuk menentukan indeks tengah array. Indeks tengah digunakan sebagai batas antara bagian kiri dan bagian kanan.

```
sort_1015(arr, l, m);
```

Perintah ini digunakan untuk mengurutkan bagian kiri array, yaitu dari indeks l sampai indeks m.

```
sort_1015(arr, m + 1, r);
```

Perintah ini digunakan untuk mengurutkan bagian kanan array, yaitu dari indeks m + 1 sampai indeks r.

```
merge_1015(arr, l, m, r);
```

Setelah bagian kiri dan kanan selesai diurutkan, method merge_1015 dipanggil untuk menggabungkan kedua bagian tersebut menjadi satu bagian yang terurut.

4. Method printArray_1015

```
static void printArray_1015(int arr[]) {
```

Method printArray_1015 digunakan untuk menampilkan isi array ke layar.

```
int n = arr.length;
```

Variabel n digunakan untuk menyimpan panjang array.

```
for (int i = 0; i < n; i++) {
```

```
System.out.print(arr[i] + " "); }
```

Perulangan ini digunakan untuk mencetak setiap elemen array satu per satu.

```
System.out.println();
```

Perintah ini digunakan untuk membuat baris baru setelah semua elemen array ditampilkan.

5. Method main

```
public static void main(String args[]) {
```

Method main adalah method utama yang pertama kali dijalankan saat program dieksekusi.

```
int arr[] = {12, 11, 13, 5, 6, 7};
```

Bagian ini digunakan untuk mendeklarasikan array yang berisi data awal yang akan diurutkan.

```
System.out.println("Sebelum terurut:");
```

```
printArray_1015(arr);
```

Bagian ini digunakan untuk menampilkan data sebelum dilakukan proses pengurutan.

```
mergeSort_2511531015 ob = new mergeSort_2511531015();
```

Perintah ini digunakan untuk membuat objek dari class mergeSort_2511531015. Objek ini diperlukan karena method sort_1015 dan merge_1015 tidak dideklarasikan sebagai static.

```
ob.sort_1015(arr, 0, arr.length - 1);
```

Perintah ini digunakan untuk memanggil method sort_1015 agar array dapat diurutkan. Parameter 0 menunjukkan indeks awal array, sedangkan arr.length - 1 menunjukkan indeks terakhir array.

```
System.out.println("\nSesudah terurut menggunakan Merge Sort:");
```

```
printArray_1015(arr);
```

Setelah proses pengurutan selesai, array yang sudah terurut ditampilkan kembali ke layar.

```
Sebelum terurut:
12 11 13 5 6 7

Sesudah terurut menggunakan Merge Sort:
5 6 7 11 12 13
```

KODE PROGRAM GUI

Program ini merupakan aplikasi GUI berbasis **Java Swing** yang digunakan untuk memvisualisasikan proses pengurutan data menggunakan metode **Bubble Sort**. Program memungkinkan pengguna memasukkan beberapa angka yang dipisahkan dengan koma, kemudian menampilkan angka tersebut dalam bentuk kotak-kotak label. Proses pengurutan dapat dilakukan secara bertahap dengan menekan tombol **Step**, sehingga pengguna dapat melihat proses perbandingan dan pertukaran elemen satu per satu.

1. Deklarasi Package dan Library

```
package pekan8_2511531015;
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.ActionEvent;
```

```
import java.awt.event.ActionListener;
```

Bagian ini menunjukkan bahwa program berada di dalam package `pekan8_2511531015`.

Library `javax.swing.*` digunakan untuk membuat komponen GUI seperti `JFrame`, `JButton`, `JTextField`, `JTextArea`, `JPanel`, dan `JLabel`.

Library `java.awt.*` digunakan untuk mengatur tampilan GUI, seperti layout, warna, ukuran, dan font. Sedangkan `java.awt.event.*` digunakan untuk menangani aksi ketika tombol diklik oleh pengguna.

2. Deklarasi Class

```
public class mergeSortGUI_2511531015 extends JFrame {
```

Class `mergeSortGUI_2511531015` merupakan class utama dari program GUI. Class ini mewarisi `JFrame`, sehingga class ini dapat berfungsi sebagai jendela utama aplikasi.

Walaupun nama class menggunakan kata `mergeSortGUI`, proses sorting yang digunakan di dalam program adalah **Bubble Sort**, karena prosesnya membandingkan dua elemen bersebelahan dan menukarnya jika urutannya salah.

3. Deklarasi Komponen GUI

```
private JTextField inputField;
```

```
private JButton setButton;
```

```
private JButton stepButton;
```

```
private JButton resetButton;
```

```
private JTextArea stepArea;
```

```
private JPanel panelArray;
```

Bagian ini digunakan untuk mendeklarasikan komponen-komponen GUI.

inputField digunakan sebagai tempat pengguna memasukkan angka. setButton digunakan untuk mengubah input menjadi array. stepButton digunakan untuk menjalankan proses sorting satu langkah demi satu langkah. resetButton digunakan untuk menghapus data dan mengembalikan tampilan ke kondisi awal.

stepArea digunakan untuk menampilkan keterangan setiap langkah sorting, sedangkan panelArray digunakan untuk menampilkan elemen array dalam bentuk label.

4. Deklarasi Variabel Array dan Kontrol Sorting

```
private int[] array;
```

```
private JLabel[] labelArray;
```

```
private int i = 0;
```

```
private int j = 0;
```

```
private int stepCount = 1;
```

```
private boolean sorting = false;
```

Variabel array digunakan untuk menyimpan angka yang dimasukkan oleh pengguna. Variabel labelArray digunakan untuk menyimpan tampilan setiap elemen array dalam bentuk JLabel.

Variabel i dan j digunakan sebagai indeks dalam proses Bubble Sort. Variabel stepCount digunakan untuk menghitung jumlah langkah sorting yang sudah dilakukan. Variabel sorting digunakan sebagai penanda apakah proses sorting sedang berlangsung atau tidak.

5. Constructor mergeSortGUI_2511531015

```
public mergeSortGUI_2511531015() {
```

Constructor ini dijalankan ketika objek GUI dibuat. Di dalam constructor ini, program mengatur tampilan utama aplikasi, ukuran jendela, posisi jendela, layout, serta menambahkan komponen-komponen GUI.

```
setTitle("Visualisasi Bubble Sort");
```

```
setSize(800, 500);
```

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
setLocationRelativeTo(null);
```

```
setLayout(new BorderLayout(10, 10));
```

Perintah setTitle digunakan untuk memberikan judul pada jendela aplikasi. setSize mengatur ukuran jendela menjadi 800 x 500 piksel. setDefaultCloseOperation membuat program berhenti ketika jendela ditutup. setLocationRelativeTo(null) membuat posisi jendela muncul di tengah layar. setLayout digunakan untuk mengatur tata letak komponen menggunakan BorderLayout.

6. Panel Input

```
JPanel inputPanel = new JPanel(new FlowLayout());
```

```
JLabel inputLabel = new JLabel("Masukkan angka (pisahkan dengan koma):");
```

```
inputField = new JTextField(30);
```

```
setButton = new JButton("Set Array");
```

```
stepButton = new JButton("Step");
```

```
resetButton = new JButton("Reset");
```

```
stepButton.setEnabled(false);
```

Bagian ini membuat panel input yang berisi label, kotak input, dan tiga tombol. Tombol **Step** awalnya dibuat tidak aktif menggunakan setEnabled(false) karena proses sorting belum bisa dilakukan sebelum pengguna memasukkan data.

```
inputPanel.add(inputLabel);
```

```
inputPanel.add(inputField);
```

```
inputPanel.add(setButton);
```

```
inputPanel.add(stepButton);
```

```
inputPanel.add(resetButton);
```

```
add(inputPanel, BorderLayout.NORTH);
```

Komponen-komponen tersebut kemudian dimasukkan ke dalam inputPanel, lalu panel tersebut diletakkan pada bagian atas jendela menggunakan BorderLayout.NORTH.

7. Panel Tampilan Array

```
panelArray = new JPanel(new FlowLayout());
```

```
panelArray.setPreferredSize(new Dimension(750, 120));
```

```
add(panelArray, BorderLayout.CENTER);
```

panelArray digunakan untuk menampilkan isi array. Setiap angka akan ditampilkan sebagai kotak label. Panel ini diletakkan pada bagian tengah jendela.

8. Area Keterangan Langkah Sorting

```
stepArea = new JTextArea();  
stepArea.setEditable(false);  
stepArea.setFont(new Font("Monospaced", Font.PLAIN, 14));  
JScrollPane scrollPane = new JScrollPane(stepArea);  
scrollPane.setPreferredSize(new Dimension(750, 250));  
add(scrollPane, BorderLayout.SOUTH);
```

stepArea digunakan untuk menampilkan proses sorting dalam bentuk teks. Komponen ini dibuat tidak dapat diedit menggunakan `setEditable(false)` karena hanya berfungsi sebagai output.

JScrollPane digunakan agar area teks dapat digulir jika isi langkah sorting sudah terlalu banyak. Bagian ini diletakkan pada bagian bawah jendela.

9. Event Tombol

```
setButton.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        setArrayFromInput();  
    }  
});
```

Ketika tombol **Set Array** ditekan, program akan memanggil method `setArrayFromInput()`. Method ini berfungsi mengambil input dari pengguna dan mengubahnya menjadi array integer.

```
stepButton.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        performStep();  
    }  
});
```

Ketika tombol **Step** ditekan, program akan memanggil method `performStep()`. Method ini menjalankan satu langkah proses sorting.

```

resetButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        reset();
    }
});

```

Ketika tombol **Reset** ditekan, program akan memanggil method `reset()`, yaitu untuk menghapus input, tampilan array, dan catatan langkah sorting.

10. Method `setArrayFromInput`

```
private void setArrayFromInput() {
```

Method ini digunakan untuk membaca angka dari kotak input. Angka yang dimasukkan pengguna harus dipisahkan dengan koma, misalnya:

10, 5, 8, 3, 1

```
String text = inputField.getText().trim();
```

Perintah ini mengambil teks dari input dan menghapus spasi di awal serta akhir.

```
String[] parts = text.split(",");
array = new int[parts.length];
```

Input kemudian dipisahkan berdasarkan tanda koma dan disimpan ke dalam array string `parts`. Setelah itu, dibuat array integer dengan panjang sesuai jumlah data.

```
array[k] = Integer.parseInt(parts[k].trim());
```

Setiap data string diubah menjadi integer menggunakan `Integer.parseInt`.

Jika pengguna memasukkan data selain angka, maka program akan menampilkan pesan error menggunakan `JOptionPane`.

11. Menampilkan Array ke GUI

```
labelArray = new JLabel[array.length];
```

Program membuat array label untuk menampilkan setiap elemen angka.

```
labelArray[k] = new JLabel(String.valueOf(array[k]));
```

```
labelArray[k].setFont(new Font("Arial", Font.BOLD, 24));
```

```
labelArray[k].setOpaque(true);
```

```
labelArray[k].setBackground(Color.WHITE);
```

```
labelArray[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
```

```
labelArray[k].setPreferredSize(new Dimension(50, 50));
```

labelArray[k].setHorizontalAlignment(SwingConstants.CENTER); Setiap angka ditampilkan dalam bentuk JLabel. Label diberi ukuran, font, warna latar putih, garis tepi hitam, dan posisi teks di tengah. Setelah itu, label dimasukkan ke dalam panelArray.

12. Method performStep

```
private void performStep() {
```

Method ini merupakan bagian utama dari proses sorting. Method ini menjalankan proses Bubble Sort secara bertahap setiap kali tombol **Step** ditekan.

```
if (!sorting || i >= array.length - 1) {
```

```
    sorting = false;
```

```
    stepButton.setEnabled(false);
```

```
    JOptionPane.showMessageDialog(this, "Sorting selesai!");
```

```
    return;
```

```
}
```

Bagian ini mengecek apakah proses sorting sudah selesai. Jika selesai, tombol **Step** akan dinonaktifkan dan muncul pesan bahwa sorting selesai.

```
labelArray[j].setBackground(Color.CYAN);
```

```
labelArray[j + 1].setBackground(Color.CYAN);
```

Dua elemen yang sedang dibandingkan diberi warna biru muda. Hal ini membantu pengguna melihat elemen mana yang sedang diproses.

```
if (array[j] > array[j + 1]) {
```

Bagian ini membandingkan dua elemen yang bersebelahan. Jika elemen kiri lebih besar dari elemen kanan, maka kedua elemen akan ditukar.

```
int temp = array[j];
```

```
array[j] = array[j + 1];
```

```
array[j + 1] = temp;
```

Kode ini digunakan untuk melakukan pertukaran nilai. Variabel temp digunakan sebagai tempat penyimpanan sementara.

Jika terjadi pertukaran, label akan diberi warna merah. Jika tidak terjadi pertukaran, program hanya mencatat bahwa tidak ada pertukaran pada langkah tersebut.

13. Menampilkan Log Langkah Sorting

```
stepLog.append("Langkah ").append(stepCount).append(": ");  
stepLog.append("Hasil: ").append(arrayToString(array)).append("\n\n");  
stepArea.append(stepLog.toString());
```

Bagian ini digunakan untuk menampilkan keterangan langkah sorting ke dalam stepArea. Informasi yang ditampilkan meliputi nomor langkah, apakah terjadi pertukaran atau tidak, serta hasil array setelah langkah tersebut.

14. Update Tampilan Array

```
private void updateLabels() {  
    for (int k = 0; k < array.length; k++) {  
        labelArray[k].setText(String.valueOf(array[k]));  
    }  
}
```

Method updateLabels() digunakan untuk memperbarui tampilan label setelah terjadi perubahan posisi elemen pada array.

15. Method resetHighlights

```
private void resetHighlights() {  
    for (JLabel label : labelArray) {  
        label.setBackground(Color.WHITE);  
    }  
}
```

Method ini digunakan untuk mengembalikan warna semua label menjadi putih sebelum proses perbandingan berikutnya dilakukan.

16. Method reset

```
private void reset() {
```

Method reset() digunakan untuk mengembalikan program ke kondisi awal. Input akan dikosongkan, panel array dihapus, catatan langkah sorting dikosongkan, dan tombol Step dinonaktifkan.

```
    sorting = false;
```

```
    i = 0;
```

```
    j = 0;
```

```
    stepCount = 1;
```

Variabel kontrol sorting juga dikembalikan ke nilai awal.

17. Method arrayToString

```
private String arrayToString(int[] arr) {
```

Method ini digunakan untuk mengubah isi array menjadi bentuk teks. Method ini diperlukan agar hasil array dapat ditampilkan di stepArea.

Contohnya, array:

1 3 5 8

akan ditampilkan menjadi:

1, 3, 5, 8

18. Method main

```
public static void main(String[] args) {
```

```
    SwingUtilities.invokeLater(new Runnable() {
```

```
        @Override
```

```
        public void run() {
```

```
            new mergeSortGUI_2511531015().setVisible(true);
```

```
        }
```

```
    });
```

```
}
```

Method main adalah method utama yang pertama kali dijalankan saat program dieksekusi.

SwingUtilities.invokeLater digunakan agar pembuatan dan tampilan GUI dijalankan pada thread khusus Swing, yaitu **Event Dispatch Thread**. Hal ini merupakan cara yang baik dalam pembuatan program GUI menggunakan Java Swing.

Perintah:

```
new mergeSortGUI_2511531015().setVisible(true);
```

digunakan untuk membuat objek GUI dan menampilkannya ke layar.

Output:

